



Using Hibernate

1. Using hibernate framework
2. Hibernate configuration file
3. Specifying mapping information
4. Hibernate components
5. Save operation
6. Update and delete operation

1. Using hibernate framework

- In order to setup the Hibernate framework for a java application, set of jar files are required because of transitive dependencies
- Along with hibernate jar files, JDBC driver jar file for selected database is even required.
- These jar files can be downloaded and added on the classpath of the application.
- It can be done even with the help of any build tool like Maven. Maven expects to add dependencies in pom.xml file

- One of the most required configuration file in Hibernate is hibernate.cfg.xml file.
- Location of the file is src/main/resource folder in maven projects
- It mainly consists of database configuration information.
- There can be multiple session factories defined in a single hibernate.cfg.xml file.
- Session factories mainly include:
 - Configuration properties: database connection related information
 - Mapping properties: this will mainly include information about mapping entities or mapping resources.
 - And many other...

- A mapping information helps Hibernate in determining how to pull the values from the classes and map them with table and associated fields.
- Previously applications had used XML files for specifying mapping information
- Now because of Jakarta persistence API, this information is written as bean entity classes with annotations.
 - This gives advantage of porting the same mapping information across multiple ORM Mapping tools.
 - It even eliminates the need of maintaining separate xml file for every entity

- JPA is a specification which provides standardization for all ORM Mapping tools
- List of very basic annotations required in every entity mapping:
 - @Entity
 - @Table
 - @Id
 - @Column

- [SessionFactory \(org.hibernate.SessionFactory\)](#)
- A session factory is a central component in Hibernate that manages sessions and provides a consistent way to interact with the database.
- [Session \(org.hibernate.Session\)](#)
- It is responsible for interacting with the database, fetching data, means virtually for each dialog with the database generally one session is created
- [Transaction \(org.hibernate.Transaction\)](#)
- A single-threaded, short-lived object used by the application to isolate the application from the underlying transaction
- [Hibernate workflow](#)
- hibernate connects with the database itself and executes the queries, then maps the results to java objects.

- Create SessionFactory instance
StandardServiceRegistry registry = new
StandardServiceRegistryBuilder().configure().build();
Metadata metadata = new
MetadataSources(registry).getMetadataBuilder().build();
SessionFactory sf =
metadata.getSessionFactoryBuilder().build();
- Open session and save the newly created instance
Emp e = new Emp(120,"Raju",34500.00f,"Admin");
Session session = sf.openSession();
Transaction tr = session.beginTransaction();
//object e to be saved as record
session.save(e);
tr.commit();

- Hibernate Session provide different methods to fetch objects from database. Two of them are – get() and load().
- These methods help in getting the persistent instances from the database which can be updated and deleted and by using appropriate methods in hibernate session it can be reflected in the database accordingly.

Updating entities	Deleting entities
<pre>Emp e = session.get(Emp.class, 102); e.setSalary(55900.00f); Transaction tr = session.beginTransaction(); session.update(e); tr.commit();</pre>	<pre>Emp e = session.get(Emp.class, 110); Transaction tr = session.beginTransaction(); session.delete(e); tr.commit();</pre>